head is just like the policy network (actor) from the previous chapter, so it outputs a 2-dimensional vector representing a discrete probability distribution over the 2 possible actions. The critic outputs a single number representing the state value. The critic is denoted $v(s)$ and the actor is denoted $\pi(s)$. Remember that $\pi(s)$ returns the log probabilities for each possible action, which in our case is 2 actions.

2. While we're in the current episode

    1. Define the hyperparameter: γ (gamma, discount factor).
    2. Start a new episode, in initial state s_t.
    3. Compute the value $v(s_t)$ and store it in the list.
    4. Compute $\pi(s_t)$, store it in the list, sample, and take action a_t. Receive the new state s_{t+1} and the reward r_{t+1}. Store the reward in the list.

3. Train

    1. Initialize $R = 0$. Loop through the rewards in reverse order to generate returns: $R = r_i + \gamma * R$.
    2. Minimize the actor loss: $-1 * \gamma_t * (R - v(s_t)) * \pi(a|s)$.
    3. Minimize the critic loss: $(R - v)^2$.

4. Repeat for a new episode.

The following listing implements these steps in Python.

Listing 5.4. CartPole actor-critic model

```
import torch
from torch import nn
from torch import optim
import numpy as np
from torch.nn import functional as F
import gym
import torch.multiprocessing as mp                          1
class ActorCritic(nn.Module):                               2
    def __init__(self):
        super(ActorCritic, self).__init__()
        self.l1 = nn.Linear(4,25)
        self.l2 = nn.Linear(25,50)
        self.actor_lin1 = nn.Linear(50,2)
        self.l3 = nn.Linear(50,25)
        self.critic_lin1 = nn.Linear(25,1)
```